



C++ Thread Fundamentals

2025

André Pereira



- **Concurrency vs. Parallelism**

- **Concurrency**: Multiple tasks processing in overlapping time periods (not necessarily simultaneously)
- **Parallelism**: Multiple tasks running at the **exact same time** (requires multiple cores/processing units)

- **Threads in HPC**

- Shared-memory parallelism on multi-core architectures.
- Typically used to exploit **intra-node** parallelism (complementary to MPI for **inter-node** parallelism).



- **Why C++ Threads?**

- Standardised since **C++11**, ensuring portability.
- Direct, low-level control over thread management and synchronization.
- High-level abstractions for low-level control
- Can be combined with higher-level frameworks like **OpenMP** or **TBB** if needed.
- Fine-grain control over thread behaviour (great for **load balancing**)
 - They will be important later

A Speedrun Through C++ Threads



- **std::thread** overview
 - Pass a function, functor, or lambda to a thread constructor
 - `std::thread t([]{/* work */});`
- Thread lifecycle
 - `join()`: Blocks until the thread finishes; ensures safe cleanup.
 - `detach()`: Thread runs independently; cannot be joined later.

```
1  void worker(int id) {  
2      std::cout << "Thread " << id << " is working\n";  
3  }  
4  
5  int main() {  
6      std::thread t(worker, 1);  
7      // Must join or detach before exiting  
8      t.join();  
9      return 0;  
10 }
```

A Speedrun Through C++ Threads



- **std::thread** overview

- Pass a function, functor, or lambda to a thread constructor
 - `std::thread t([]{/* work */});`

- Thread lifecycle

- **join()**: Blocks until the thread finishes; ensures safe cleanup.
- **detach()**: Thread runs independently; cannot be joined later.

```
1  void worker(int id) {
2      std::cout << "Thread " << id << " is working\n";
3  }
4
5  int main() {
6      std::thread t(worker, 1);
7      // Must join or detach before exiting
8      t.join();
9      return 0;
10 }
```

- Forgetting to **join or detach** => `std::terminate` will be called
- Exiting main while threads are still running => undefined behaviour

A Speedrun Through C++ Threads



- Passing arguments
 - By **value**: Makes a copy.
 - By **reference**: Uses `std::ref` or capture references carefully with lambdas.
 - By **pointer**: Copies the address provided.
 - **Lambda captures**: `[&]`, `[=]`, or selective captures to control data access.
- Returning Results
 - Use **shared data** (protected by mutex or atomic operations).
 - **`std::promise` + `std::future`** to send back results or exceptions.

```
1  std::promise<int> p;  
2  std::future<int> f = p.get_future();  
3  
4  std::thread t([&p]() {  
5      int result = compute_some_value();  
6      p.set_value(result);  
7  });  
8  // ...  
9  int value = f.get(); // blocks until  
10 |         |         |         | // set_value is called  
11 t.join();
```



- **`std::mutex`** and variations
 - **`std::timed_mutex`**: Allows timeout-based lock attempts.
 - **`std::recursive_mutex`**: Can be locked multiple times by the same thread (careful!).
- Locking mechanisms
 - **RAII** (Resource Acquisition Is Initialization) with:
 - **`std::lock_guard<std::mutex> lock(mtx)`**: Simple, acquires on construction, releases on destruction.
 - **`std::unique_lock<std::mutex>`**: More flexible, can unlock/lock multiple times.
 - **`std::scoped_lock`**: C++17 feature for multiple mutexes with no deadlock.

Synchronisation Primitives - Locks



- Common pitfalls
 - **Deadlock:** Acquiring multiple locks in an inconsistent order.
 - **Double Locking:** Attempting to lock the same mutex twice from the same thread without using `std::recursive_mutex`.

```
1  std::mutex m;  
2  int sharedCounter = 0;  
3  
4  ✓ void increment() {  
5      m.lock();  
6      ++sharedCounter;  
7      m.unlock();  
8  }  
9  
10 ✓ int main() {  
11     std::thread t1(increment), t2(increment);  
12     t1.join(); t2.join();  
13     std::cout << sharedCounter << "\n";  
14 }
```

Synchronisation Primitives - Locks



- Common pitfalls
 - **Deadlock:** Acquiring multiple locks in an inconsistent order.
 - **Double Locking:** Attempting to lock the same mutex twice from the same thread without using `std::recursive_mutex`.

```
1  std::mutex m;  
2  int sharedCounter = 0;  
3  
4  void increment() {  
5      std::lock_guard<std::mutex> lock(m);  
6      ++sharedCounter;  
7  }  
8  
9  int main() {  
10     std::thread t1(increment), t2(increment);  
11     t1.join();  
12     t2.join();  
13     std::cout << "Counter = " << sharedCounter << "\n";  
14 }
```

Synchronisation Primitives – Condition Variables



- **TBC**